IT4931 / IT5384 / IT4043E · SOICT · HUST

Big Data Architecture

Lecture 09 — Lambda & Kappa

Batch + stream patterns for massive data

Learning outcomes

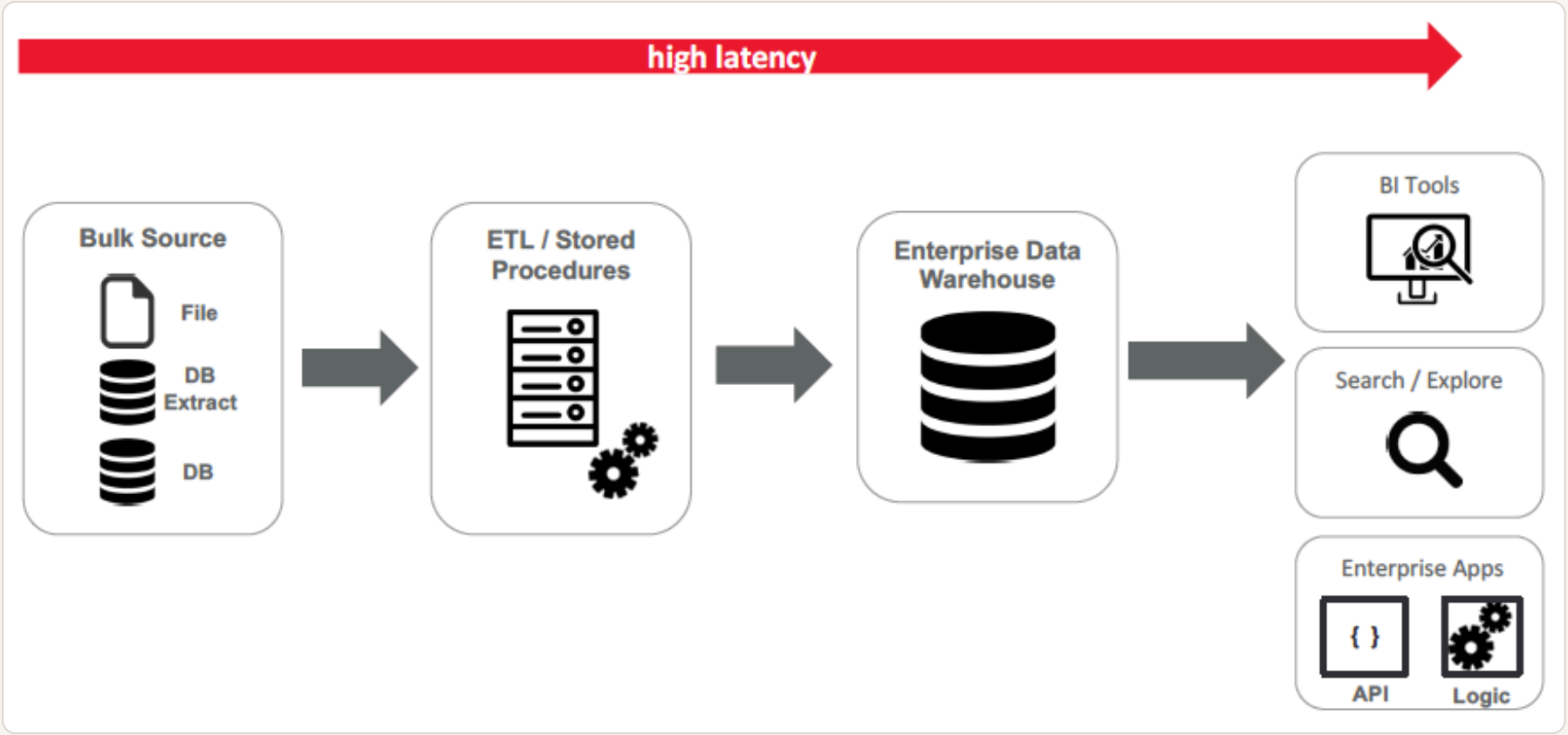
1. **Motivate** Lambda from traditional BI limits and batch vs stream trade-offs
2. **Explain** Lambda layers: batch · speed · serving; immutability & recompute
3. **Explain** Kappa: single stream pipeline + replay from the log
4. **Compare** Lambda vs Kappa trade-offs (when to simplify)

Roadmap

1. **9.1** Lambda architecture
2. **9.2** Kappa architecture
3. **Compare** · course links

9.1 Lambda architecture

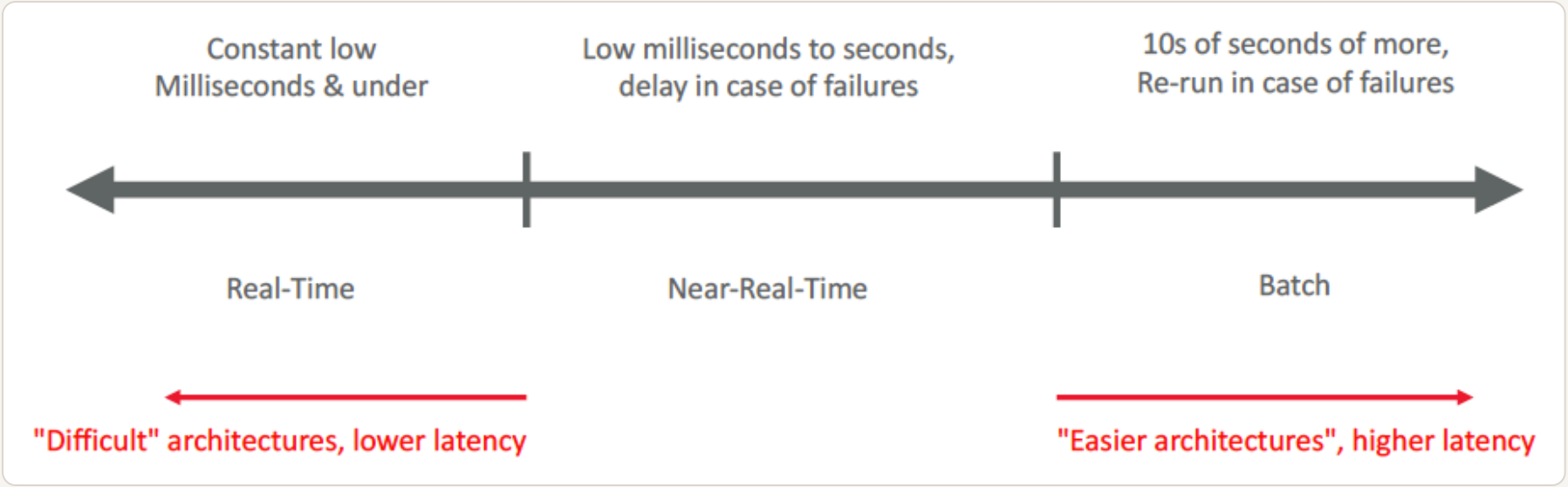
Traditional BI infrastructures



Batch vs stream

Batch
Finite, static datasets · high throughput · higher latency · complex analytics

Stream
Unbounded continuous data · low latency · near-real-time reactions



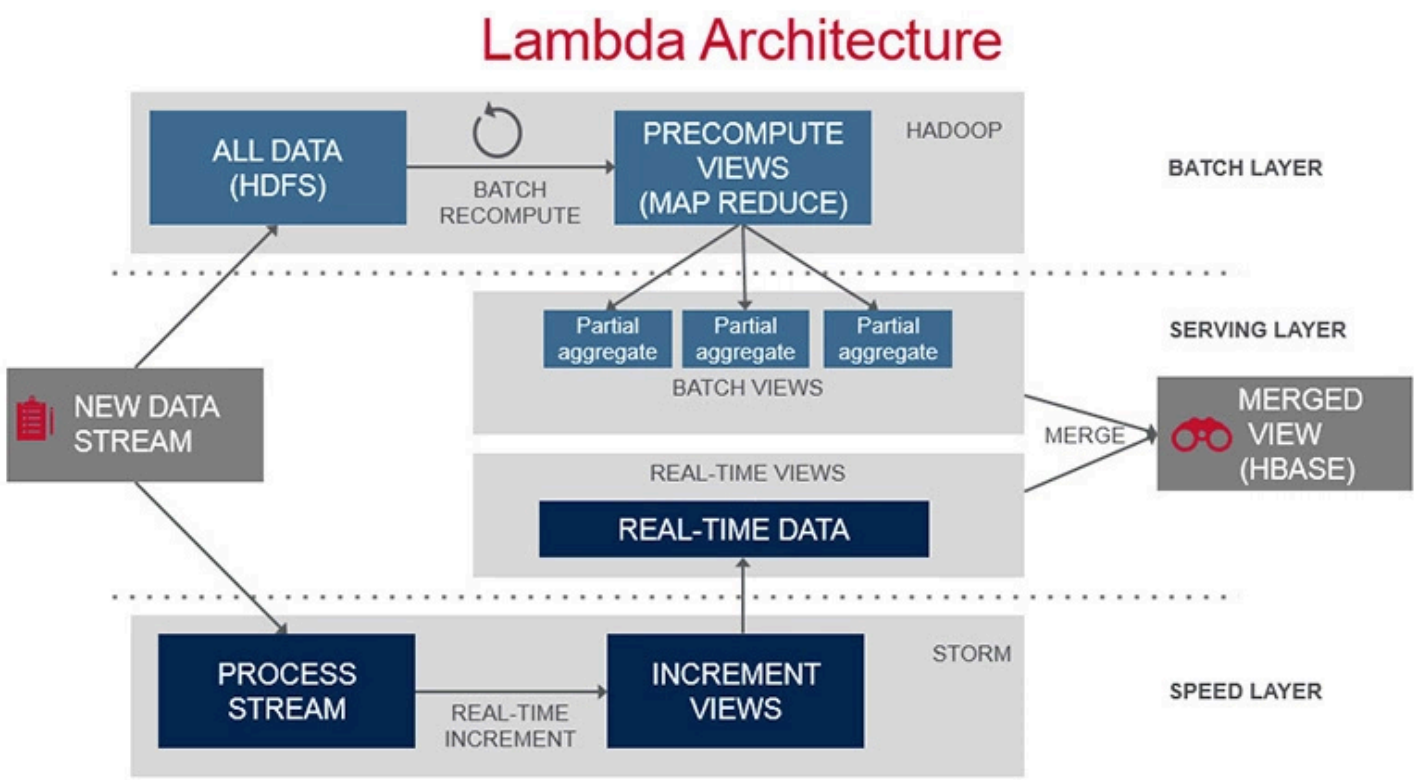
What Lambda is for

Need **fast access to historical data** *and* **real-time stream** updates in one query surface (e.g. recommendations, fraud, dashboards).

Path	What you get
Batch	Accurate views over the full master dataset (higher latency)
Speed	Fresh increments while batch is still running
Serving	Merge both → answer “history + now”

PPTX used a cartoon meme here — teach the requirement, not the joke.

Lambda layers (overview)



Batch layer

Role: manage the **master dataset** and pre-compute **batch views**.

Components	Distributed FS (HDFS/S3) · batch engines (Hadoop, Spark)
Functions	Store immutable raw data · batch processing · create batch views

Speed layer

Role: compensate for batch lag with **low-latency** increments.

Components	Stream systems (Storm, Flink, Spark Streaming, …)
Functions	Real-time processing · incremental real-time views

Serving layer

Role: answer queries by **merging** batch and real-time views.

Components	Low-latency stores (HBase, Cassandra, …)
Functions	Index batch views · fast reads · merge batch + real-time

Lambda implementation (example stack)

One **possible** classic stack — roles matter more than product logos.

Layer	Role	Example tech
Ingest	Fan-out raw events	Kafka
Batch	Immutable master · recompute views	HDFS/S3 · Spark / MapReduce
Speed	Low-latency increments	Flink / Spark Streaming / Storm
Serving	Merge views · query	HBase / Cassandra · Parquet views

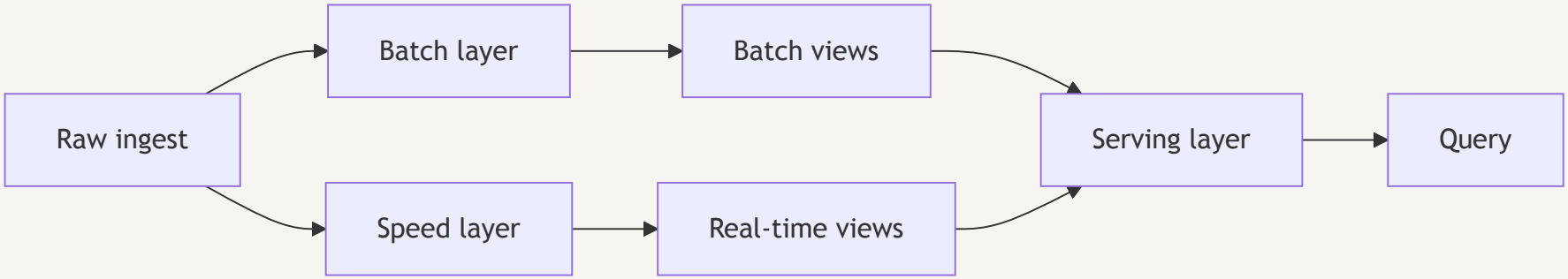
Cloud sketch (AWS example)

Same Lambda **roles** on managed services — names evolve; map the architecture.

Layer	AWS-shaped example
Ingest	Kinesis Data Firehose / Streams
Batch	S3 master · Glue / EMR batch jobs
Speed	Kinesis Data Analytics (or Flink on managed)
Serving	S3 views · Athena · QuickSight (BI)

Illustrative mapping only — not a 2026 bake-off.

Data flow in Lambda



Parallel paths; **serving** answers by combining both views.

Core principles

1. **Immutability** — append raw data; don't overwrite history
2. **Recomputation** — rebuild views from master when logic changes
3. **Separation of concerns** — batch vs speed vs serving
4. **Polyglot persistence** — right store per layer / access pattern

Fault tolerance

Mechanism	Effect
Immutable master	Raw data survives bad jobs
Distributed batch/stream	Node loss → redistribute
Dual paths	Speed serves while batch recomputes

Mitigating human errors

- Bug in view logic → **fix code**, rerun from immutable master — no lost raw data
- Batch / speed **isolated** — a speed bug needn't corrupt history
- Batch layer is the place to **test heavy** logic before it dominates serving

Facilitating reprocessing

- Immutable master always available for recompute
- Batch layer optimized to process **entire** datasets
- While batch reprocesses history, **speed keeps ingesting** new events — system stays live

Eventual consistency of views

- Batch view: **strongly consistent but lagged**
- Speed view: **fresh but approximate / partial**
- Serving merge → **eventual** agreement as batch catches up

Lambda challenges

- **Two codepaths** — duplicate logic, drift risk
- **Ops complexity** — monitor/sync two systems
- **Merge hardness** — correct serving semantics
- **Cost** — run batch + speed continuously

→ Motivation for **Kappa** simplification.

9.2 Kappa architecture

Kappa idea

One stream processing path for real-time **and** historical (via **replay**).

Motivation: drop the separate batch system; keep a durable **event log** (e.g. Kafka) as source of truth.

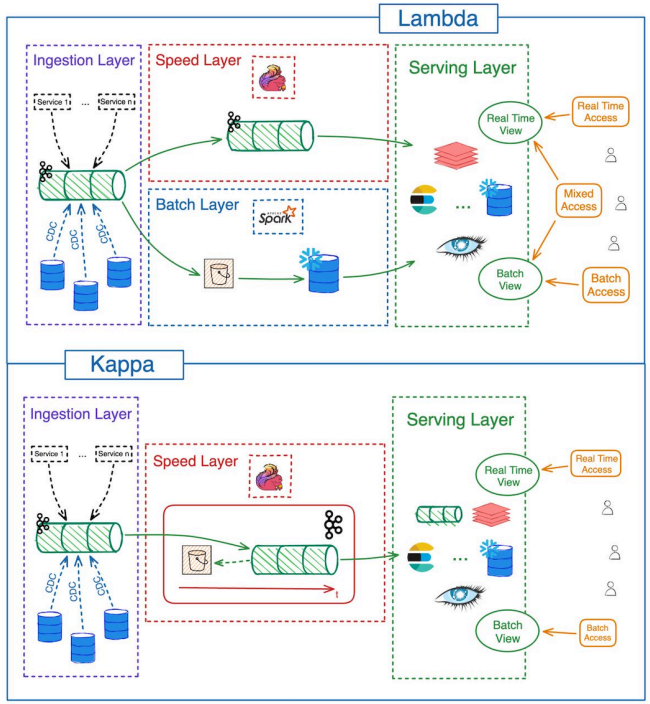
Kappa as simplification of Lambda

Lambda pain	Kappa response
Two systems to build/ops	**One** processing path
Dual code → drift	**One** codebase
Merge batch + speed views	Serving updates from the **same** pipeline
Batch job for history	**Replay** the log from offset 0

Still need mature stream processing (state, late data, exactly-once).

Kappa architecture

Top: Lambda dual path · Bottom: Kappa single stream + replay (same serving idea).



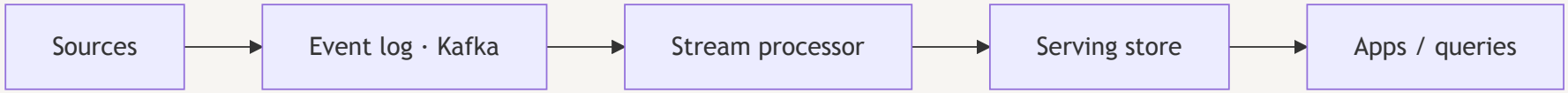
Kappa principles

Principle	Meaning
Data as **event stream**	Append-only ordered events
Replay	Reset offsets → reprocess with new logic
Single pipeline	One codebase for “batch-like” and live

Key components

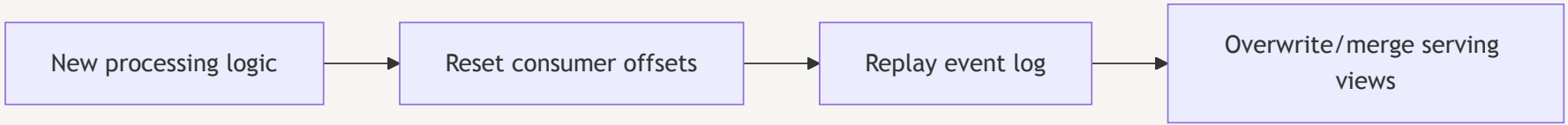
Component	Role	Examples
Event log	Durable ordered ingress	Apache Kafka
Stream processor	Stateful transforms / aggregates	Flink, Kafka Streams, Spark Streaming
Serving	Fast reads of results	Cassandra, Elasticsearch, Druid

Kappa data flow



Ingest → durable log → continuous processing → incremental serving views.

Data reprocessing in Kappa



	Lambda	Kappa
Reprocess	Separate batch job on master	Replay log with **one** codebase

Kappa advantages

- **Simpler** — one pipeline, one codebase
- **Lower dual-path inconsistency** — single source of truth (event log)
- **Natural low latency** — no wait for batch jobs
- **Scales** with the log; easier to add consumers

Kappa challenges

- **Long retention** of the full event history → storage cost
- Stream design hard: late data, exactly-once, distributed **state**
- Some “batch-like” joins / full recomputes are awkward or expensive
- Learning curve; not ideal if historical vs realtime workloads truly diverge

Lambda vs Kappa (choose)

	Lambda	Kappa
Paths	Batch + speed	Single stream
Recompute	Batch job on master	Replay log
Complexity	Higher dual system	Stream/state complexity
Fit	Clear split historical vs realtime teams/tools	Unified stream platform (Kafka + Flink/Spark)

Course links

- **L08 Kafka** — event log for Kappa / speed ingress
- **L10–L11 Spark** — batch & streaming engines inside either pattern
- **L04 Lakehouse** — today's dominant "unified" storage+compute story

Takeaways

1. **Lambda** = immutable master + batch views + speed views + serving merge
2. **Kappa** = one stream path + replay from the log
3. Pick based on **ops skill**, latency needs, and whether dual codepaths are worth it

Next

L10 — Spark batch